Лекция 36. Безопасность. Аутентификация. Авторизация. Keycloak

Аутентификация - процедура проверки подлинности.
Пользователь в подтверждение своей личности предоставляет системе что-то, чем может обладать только он (логин-пароль, токен, сертификат и т.п.)
Авторизация - процедура наделения и проверки прав на доступ к ресурсу. Сначала пользователю предоставлются права доступа к ресурсу (данным). Затем, когда пользователь запрашивает этот ресурс (данные), производится проверка: есть ли у него соответствующие права (были ли они ему предостлавены).
Существует большое число способов аутентификации клиента:

- ввод логина и пароля
- ввод кода из СМС
- ввод токена персонального доступа
- ввод биометрических данных (голос, изображение)
- и т.д.
  Вс эти способы могут применяться как поотдельности (однофакторная аутентификация) так и вместе (многофакторная аутентификация).
  Одним из фреймворков аутентификации на основе токенов, применяемых в микросервисной архитектуре, является OAuth2.0. Если говорить коротко, то OAuth2.0 позволяет:
- обменять логин + пароль пользователя на токен доступа и токен обновления,
- передавать в токене доступа (JSON Web Token, JWT) полезную информацию о пользователе и его полномочиях.
  JWT состоит из трёх частей:
  Header (заголовок) - содержит информацию об алгоритме шифрования и типе токена;
  Payload - полезные данные токена. Содержат информацию о владельце токена и его правах доступа.

Signature - подпись - значение, полученное путём шифрования предыдущих двух частей.
Существует большое количество приложений, предоставляющих реализацию OAuth2.0:

- от признанных стандартов - используются в основном средними и небольшими компаниями,
- до самодельных реализаций - используются в основном крупными и очень крупными компаниями.
  Одну из таких реализаций - Keycloak - мы рассмотрим на нашем курсе.
  Это позволит вам познакомиться с практической стороной реализации OAuth2.0 на реальном проекте.

1. Для начала нам необходимо подготовить базу данных для работы Keycloak. Ранее мы создавали базу данных для нашего учебного проекта, для чего запускали в докер-контейнере сервер PostgreSQL. Теперь нам нужно на этом же сервере создать ещё одну базу данных. Для этого зайдите в SQL-клиент запущенного в контейнере сервера PostgreSQL любым удобным вам способом (в консоли, pgAdmin, DBeaver или IDE) и создайте базу даных:
   create database keycloak_db;
2. Добавим в файл docker/docker-compose.yaml в нашем проекте конфигурацию для запуска Keycloak:

```yaml
version: "3"

volumes:
  data:
  export:
  keycloak_data:
networks:
  idm_network:

services:
  postgres:
    image: docker.io/postgres:17-alpine
    hostname: postgres-db
    ports:
      - 127.0.0.1:5432:5432
    environment:
```

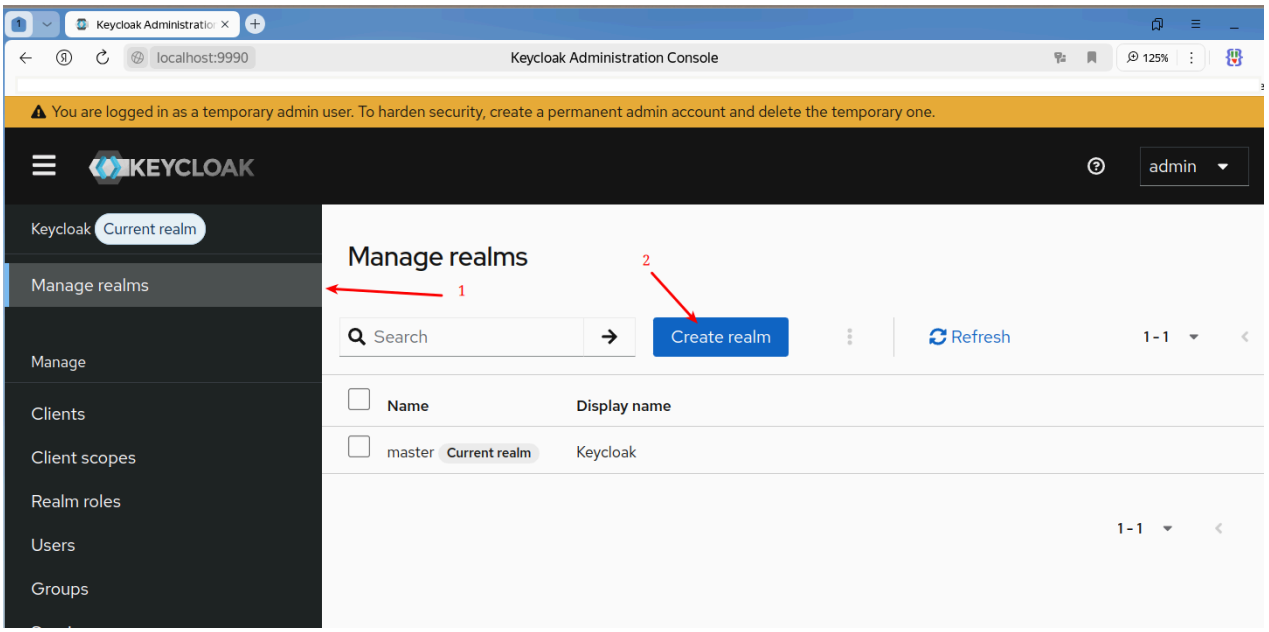
```yaml
      - POSTGRES_PASSWORD=postgres
    volumes:
      - data:/var/lib/postgresql/data
      - export:/export
    networks:
      - idm_network
    restart: unless-stopped
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: '512M'

  keycloak:
    image: quay.io/keycloak/keycloak:26.2.2
    ports:
      - "9990:8080"
      - "9991:8443"
    environment:
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin
      KC_PROXY: edge
      KC_METRICS_ENABLED: "false"
      KC_HEALTH_ENABLED: "false"
      KC_DB: postgres
      KC_DB_URL_HOST: postgres-db
      KC_DB_URL_PORT: 5432
      KC_DB_SCHEMA: public
      KC_DB_URL_DATABASE: keycloak_db
      KC_DB_USERNAME: postgres
      KC_DB_PASSWORD: postgres
      KC_LOG_LEVEL: INFO
    depends_on:
      - postgres
    command:
      - start-dev
    volumes:
      - keycloak_data:/opt/keycloak/data
      - ./auth/import:/opt/keycloak/data/import
    networks:
      - idm_network
    healthcheck:
      test: [ "CMD", "curl", "-f", "http://localhost:9990/health"
]
```

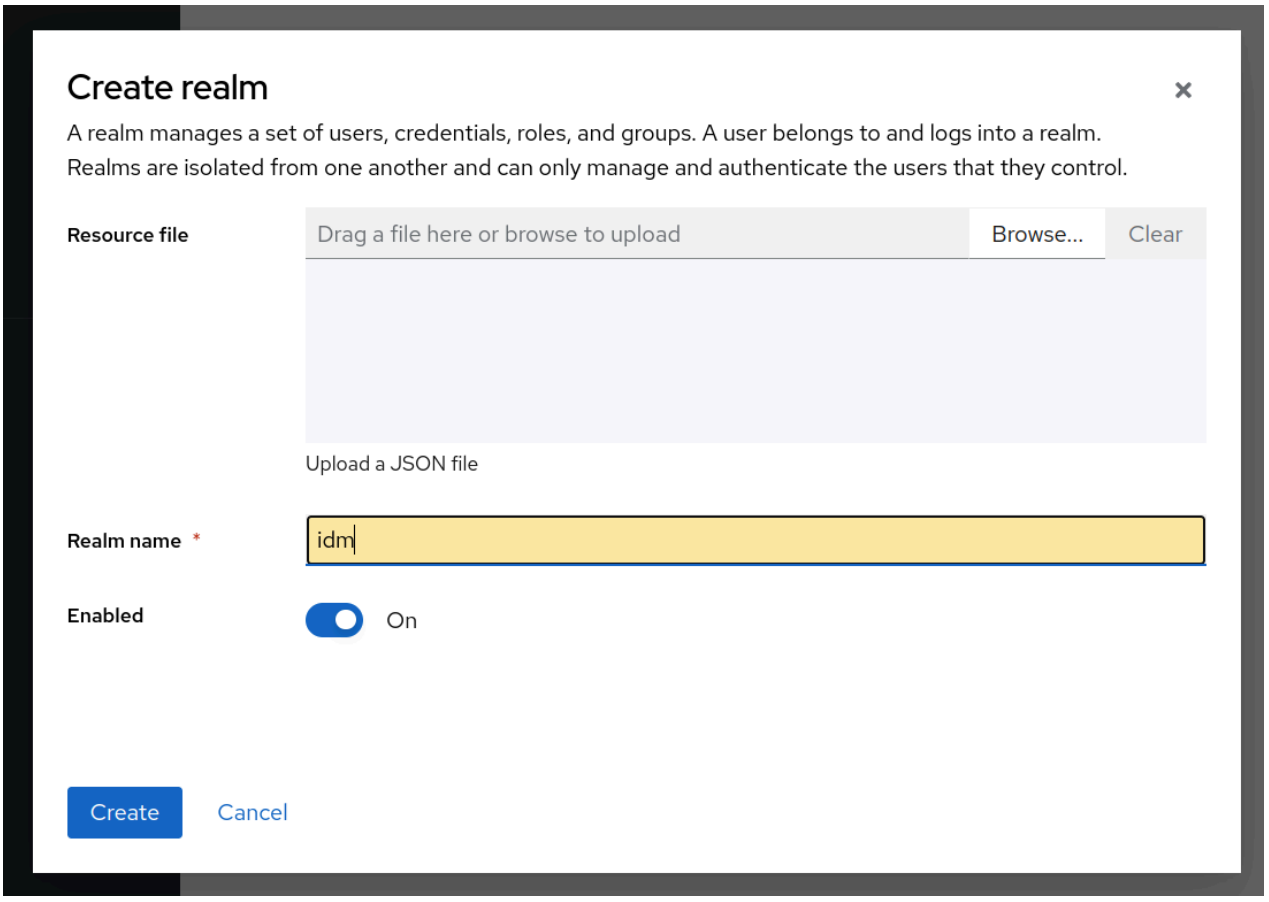
```
      interval: 30s
      timeout: 10s
      retries: 3
    restart: unless-stopped
```

1. Запустим Keycloak:
   docker compose up keycloak
   Или кнопкой в ИДЕ:

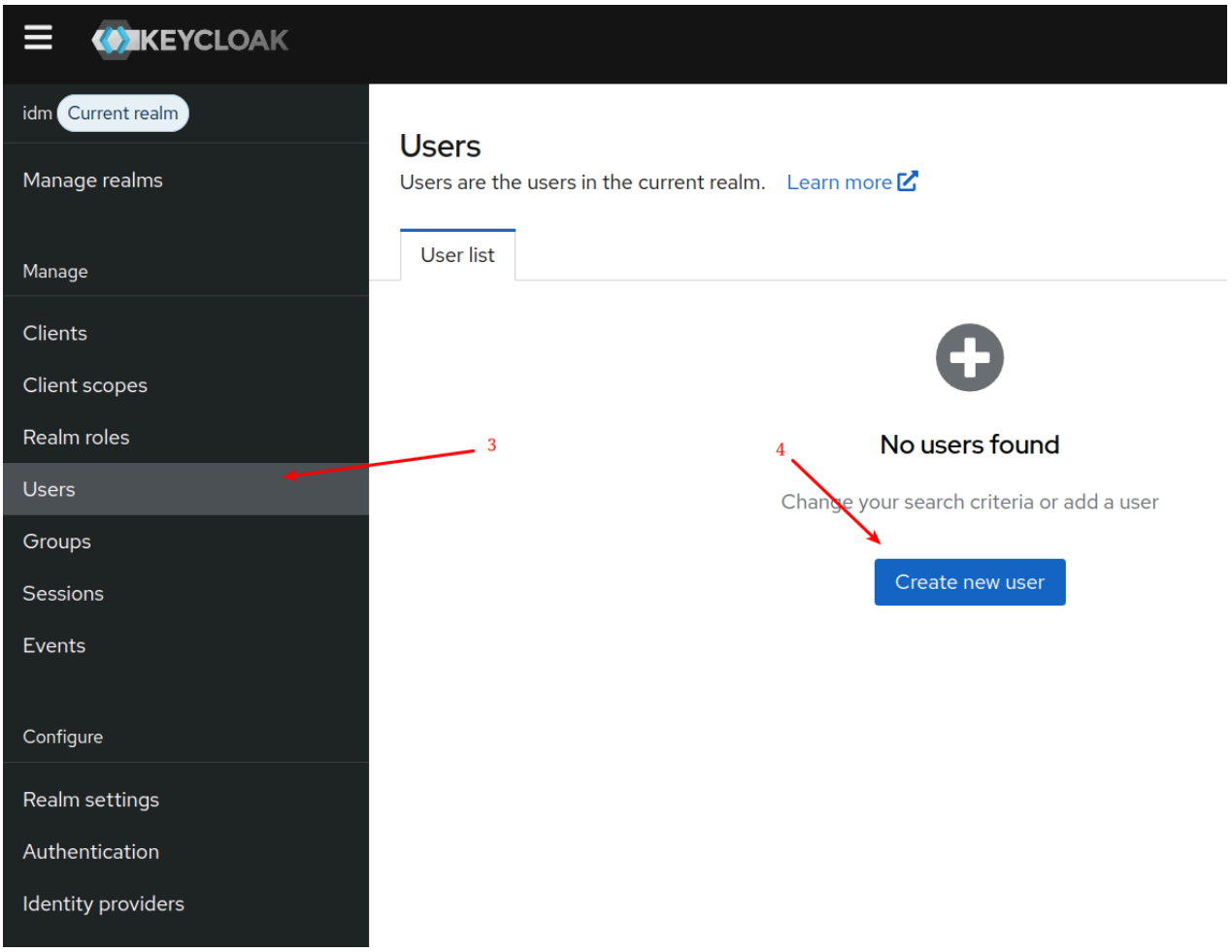
```
29
30  keycloak:
31    image: quay.io/keycloak/keycloak:26.2.2
32    ports:
33      - "9990:8080"
34      - "9991:8443"
35    environment:
36      KEYCLOAK_ADMIN: admin
37      KEYCLOAK_ADMIN_PASSWORD: admin
38      KC_PROXY: edge
```

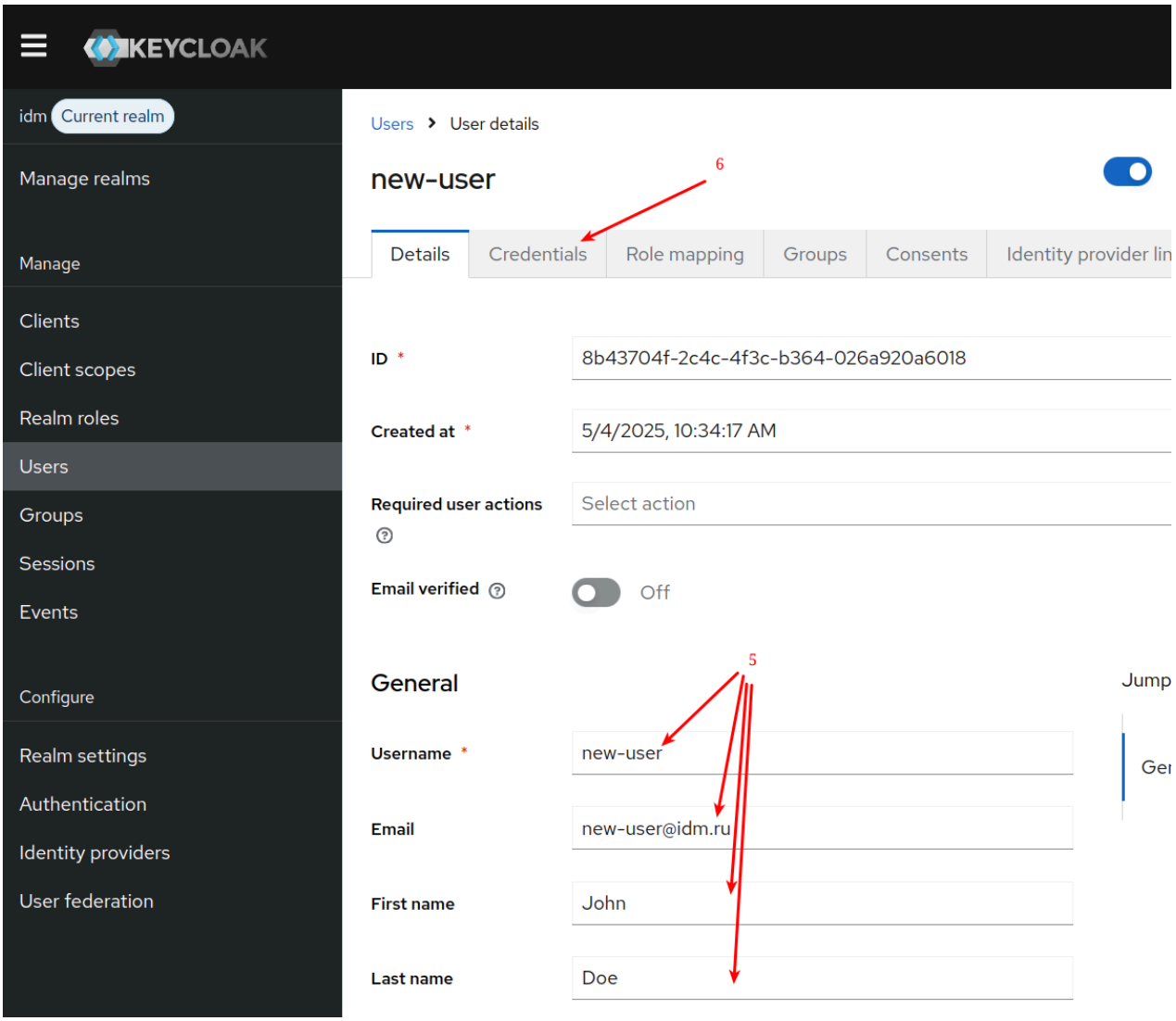
2. Зайдём в админку Keycloak:
   http://localhost:9990/admin/
   логин/пароль администратора: admin/admin
3. Создадим новый реалм (аналог: проект, рабочее пространство) и назовём его idm (1), (2):


Keycloak Administratio
localhost:9990
Keycloak Administration Console
125%
You are logged in as a temporary admin user. To harden security, create a permanent admin account and delete the temporary one.
KEYCLOAK
admin
Keycloak
Current realm
Manage realms
1
2
Manage
Clients
Client scopes
Realm roles
Users
Groups
Search
Create realm
Refresh
1 - 1
Name
Display name
master
Current realm
Keycloak
1 - 1

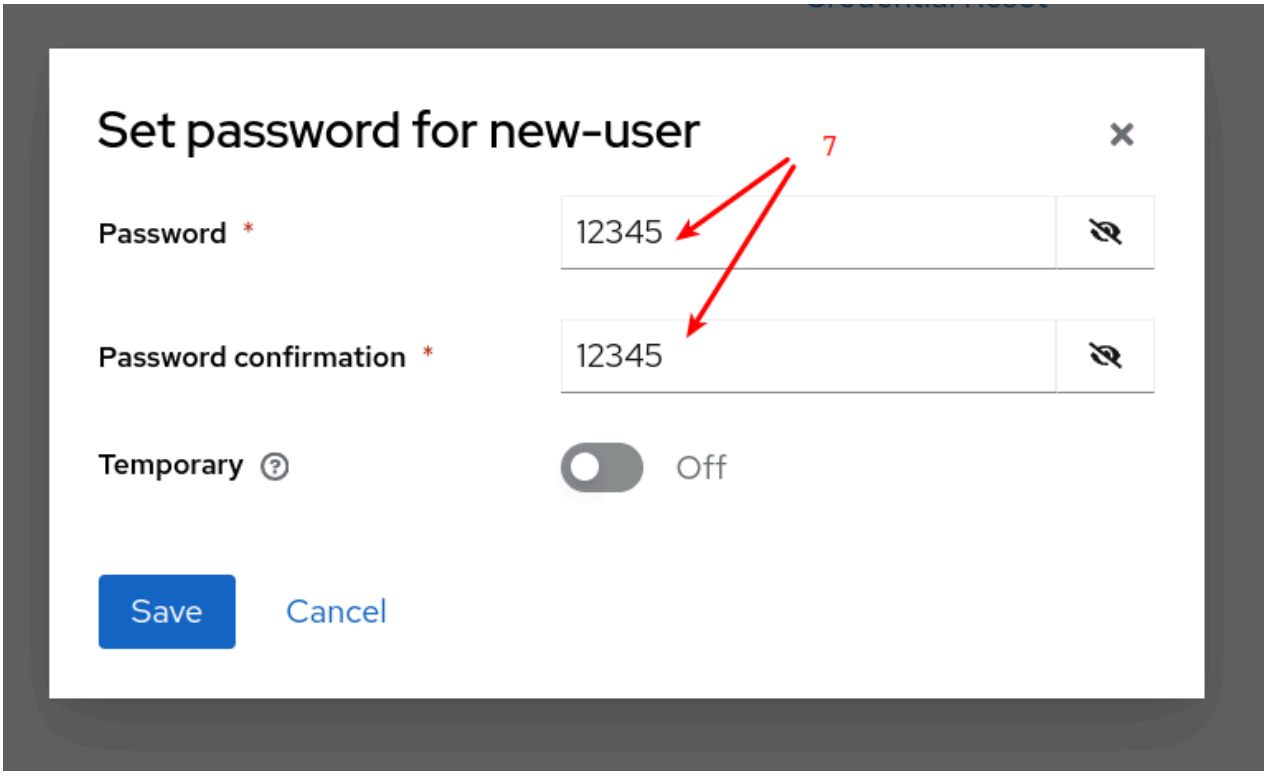

Create realm
A realm manages a set of users, credentials, roles, and groups. A user belongs to and logs into a realm.
Realms are isolated from one another and can only manage and authenticate the users that they control.
Resource file
Drag a file here or browse to upload
Browse...
Clear
Upload a JSON file
Realm name *
idm
Enabled
On
Create
Cancel

4. В этом реалме создадим нового пользователя (3), (4), (5)

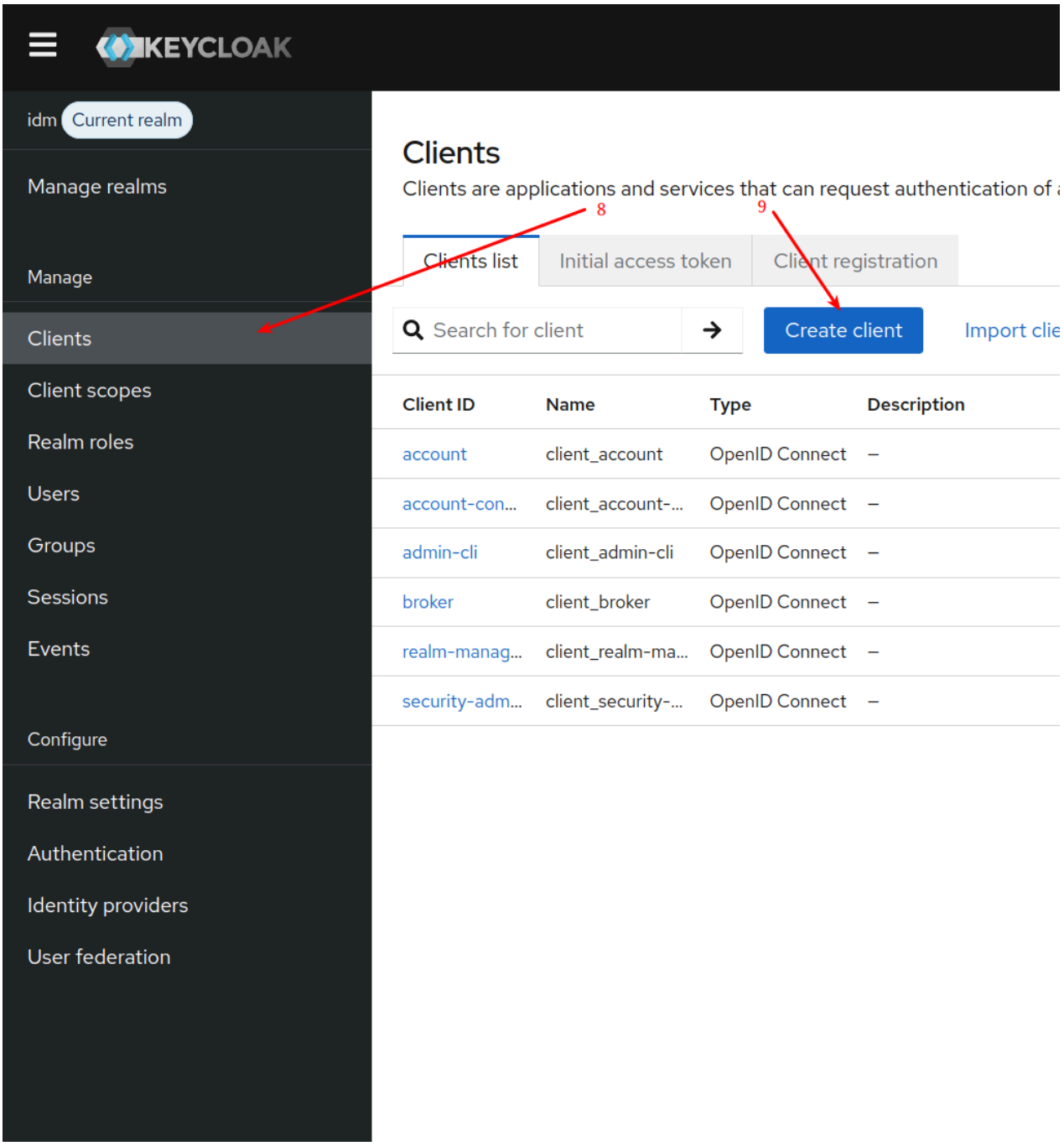




5. И зададим ему новый пароль (6), (7)



6. В этом же реалме создадим клиента (зарегистрируем наше приложение) (8), (9), (10)


KEYCLOAK
idm
Current realm
Manage realms
Manage
Clients
Client scopes
Realm roles
Users
Groups
Sessions
Events
Configure
Realm settings
Authentication
Identity providers
User federation
8
9
Clients
Clients are applications and services that can request authentication of
Clients list
Initial access token
Client registration
Search for client
Create client
Import clie
Client ID
Name
Type
Description
account
client_account
OpenID Connect
–
account-con...
client_account-...
OpenID Connect
–
admin-cli
client_admin-cli
OpenID Connect
–
broker
client_broker
OpenID Connect
–
realm-manag...
client_realm-ma...
OpenID Connect
–
security-adm...
client_security-...
OpenID Connect
–

Create client

Clients are applications and services that can request authentication of a user.

1 General settings
2 Capability config
3 Login settings

Client type: OpenID Connect

Client ID *: idmapp (10)

Name:

Description:

Always display in UI: Off

7. Создадим роль IDM_ADMIN (11), (12)

KEYCLOAK

idm Current realm

Manage realms

Manage

Clients

Client scopes

Realm roles (11)

Users

Groups

Sessions

Events

Configure

Realm roles > Create role

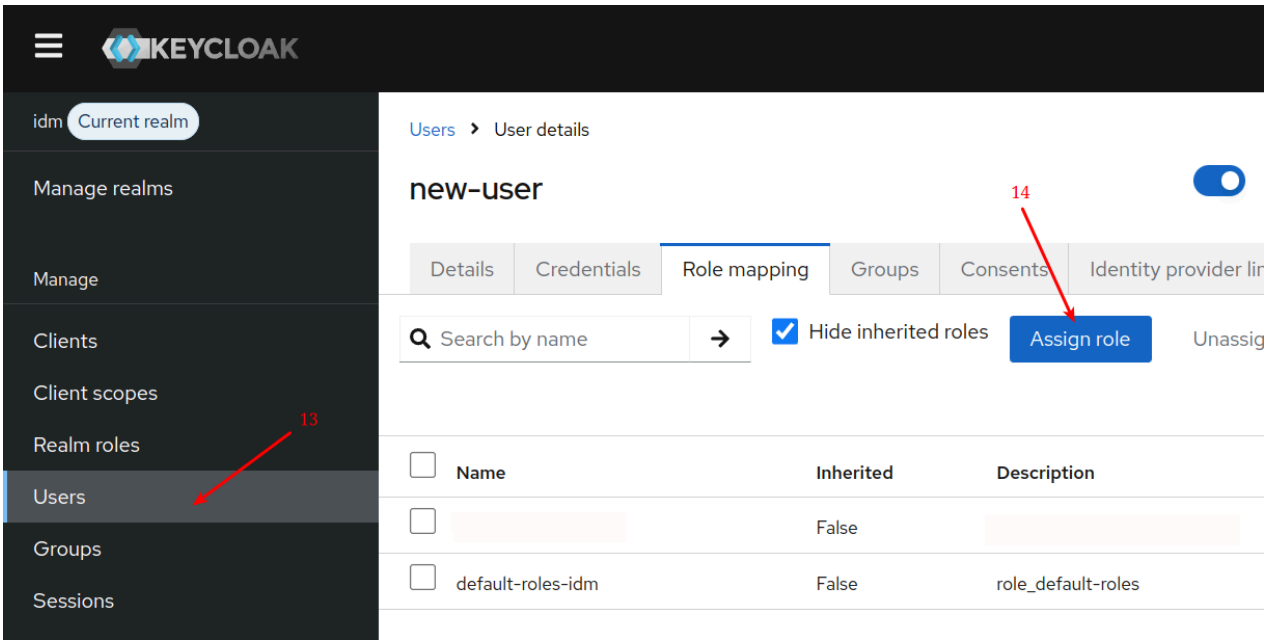
Create role

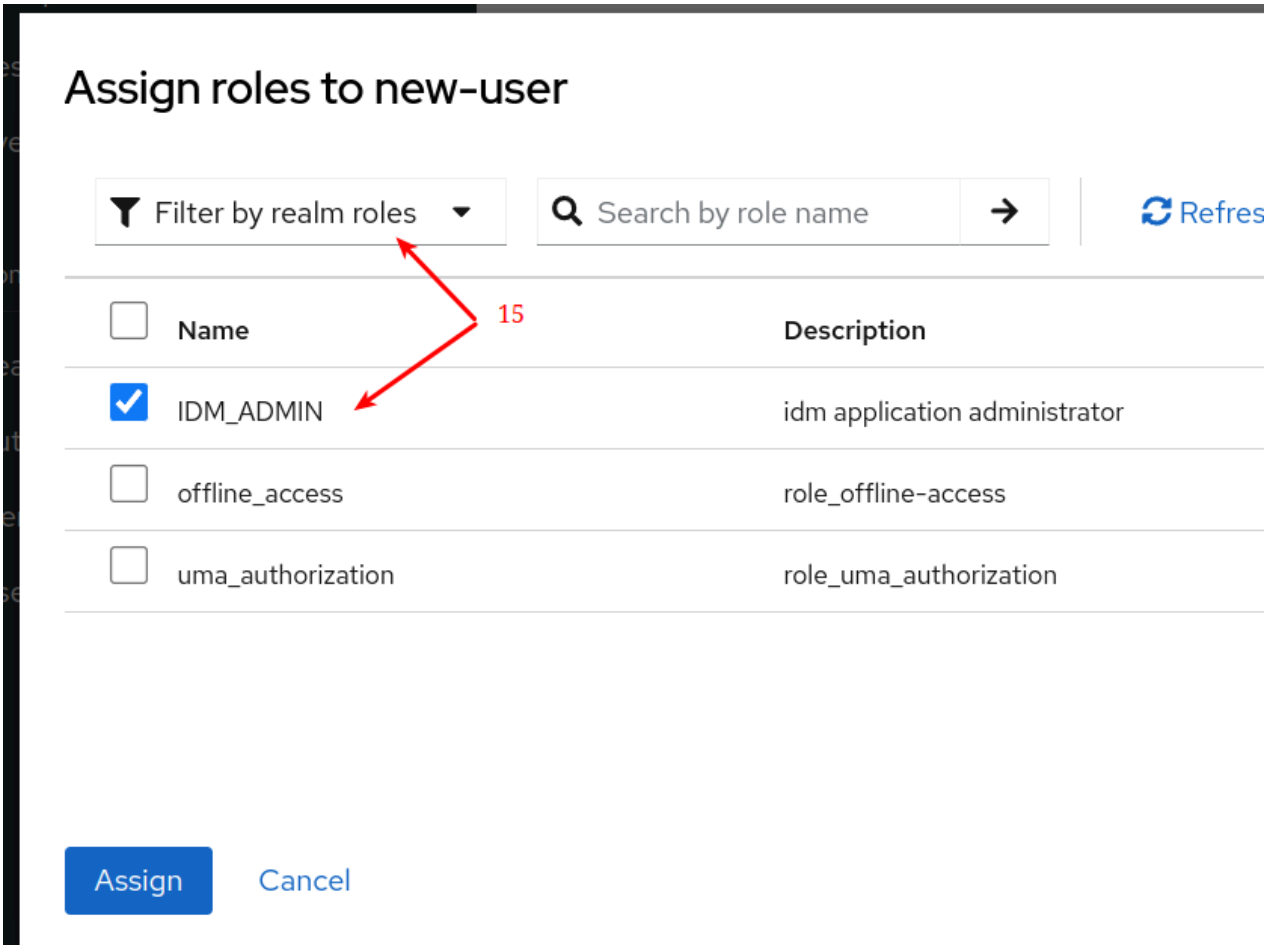
Role name *: IDM_ADMIN (12)

Description: idm application administrator

Save Cancel

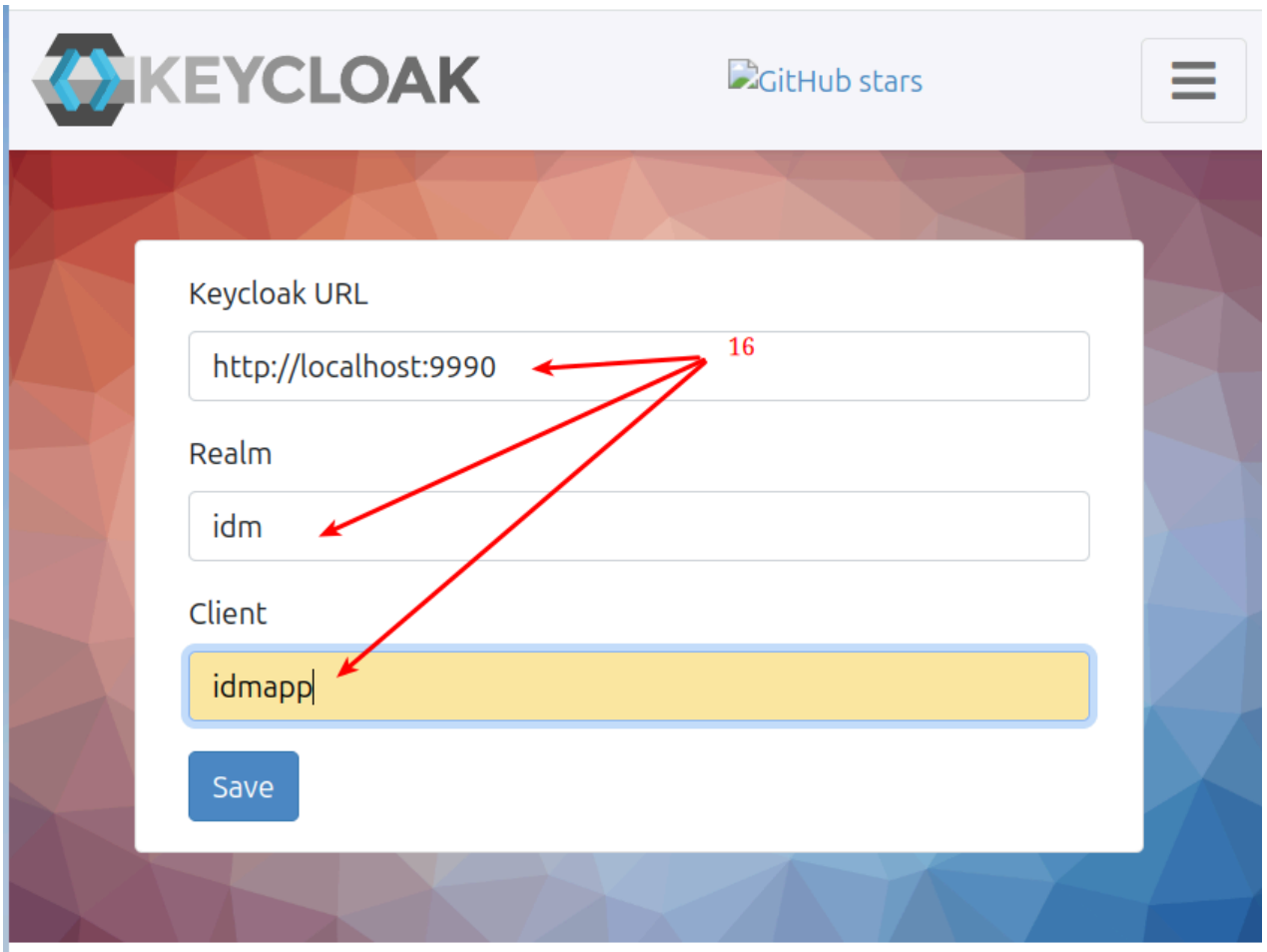
7. Назначим роль пользователю из п.5 (13), (14), (15)



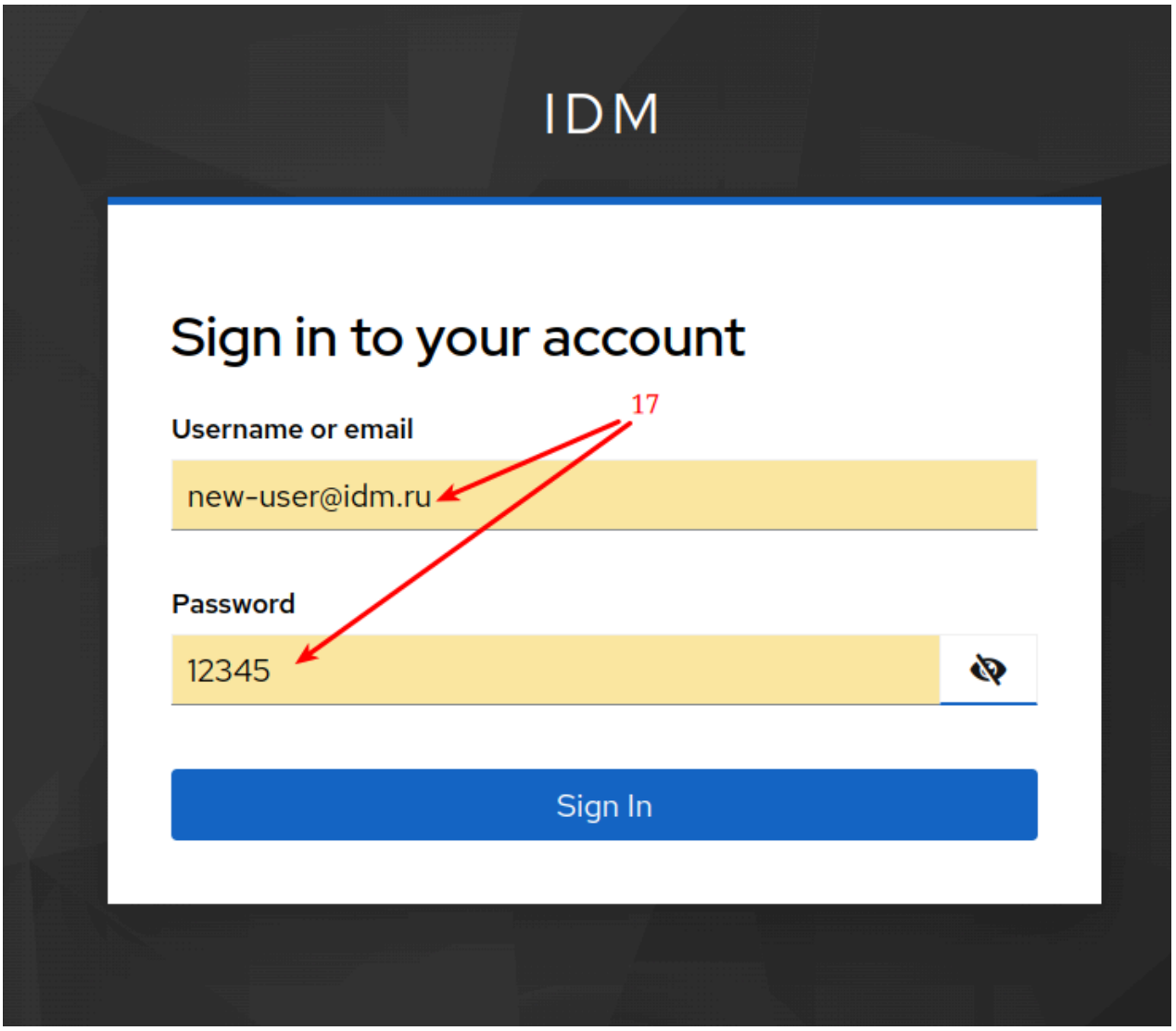


8. Зайдём на сайт Keycloak:
https://www.keycloak.org/app/

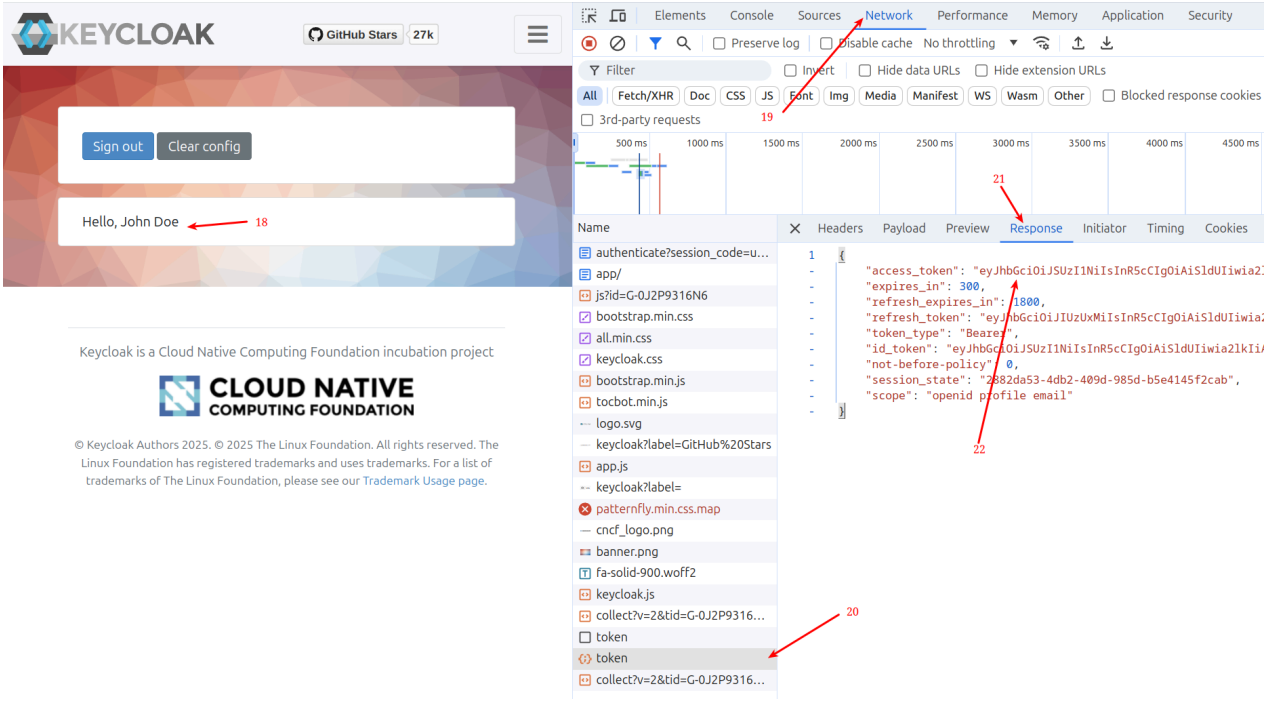
9. В настроим там страничку логина для нашего приложения (16)



10. Сделаем Sign in с учётными данными пользователя, которого мы создали в п.5 (17)



11. Увидим приветствие (18)
12. В инструментах разработчика в браузере увидим наш токен доступа - JWT (19), (20), (21), (22).



Другой способ получить токен доступа - выполнить веб-запрос:

```
~/idm$ curl --location
'http://localhost:9990/realms/idm/protocol/openid-connect/token '
\
--header 'Content-Type: application/x-www-form-urlencoded' \
-d 'client_id=idmapp' \
-d 'username=new-user@idm.ru' \
-d 'password=12345' \
-d 'grant_type=password' \
-d 'client_secret=7WwQCUqicuT7bsymH6jVo04r21OGxyQL' \
-d 'scope=openid'
```

1. Скопируем токен доступа, вставим его в левую часть формы на сайте jwt.io (23) и в правой части формы увидим декодированное содержимое токена (24). Здеь вы можете увидеть, что JWT имеет структуру JSON. В терминах JWT, ключи в его структуре называются клэймами (claims).

JWT Decoder JWT Encoder

Paste a JWT below that you'd like to decode, validate, and verify.

Generate example

ENCODED VALUE

JSON WEB TOKEN (JWT) COPY CLEAR

Valid JWT

Signature Verified


```
eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJSV2twQVZtbml0eFF2aEw
wVXhhTzJKclN2b2tsT21qbGNXUmZXRkZicVRnIn0.eyJleHAiOjE3NDYzNDU2MTUsImlhdC
I6MTc0NjM0NTMxNSwiYXV0aW1lIjoxNzQ2MzQ1MzE0LCJqdGkiOiJvbnJ0YWM6MGM0M
2Q4MTYtMjI2Ny00M2VjLWI5OWMtOTBjZDJjNmRiMWI1IiwiaXNzIjoiaHR0cDovL2xvY2Fs
aG9zdDo5OTkwL3JlYWxtcy9pZG0iLCJhdWQiOiJhY2NvdW50Iiwic3ViIjoiOGI0MzcwNGY
tMmM0Yy00ZjNjLWIzNjQtMDI2YTkyMGE2MDE4IiwidHlwIjoiQmVhcmVyIiwiYXpwIjoiaW
RtYXBwIiwic2lkIjoiNjVlZjFmOTAtMTg0OS00MTcyLTliZmMtZjcwNWE5Y2U2YTdlIiwiY
WNyIjoiMSIsImFsbG93ZWQtb3JpZ2lucyI6WyJodHRwczovL3d3dy5rZXljbG9hay5vcmci
XSwicmVhbG1fYWNjZXNzIjp7InJvbGVzIjpbIklETV9BRE1JTiIsIm9mZmxpbmVfYWNjZXN
zIiwidW1hX2F1dGhvcml6YXRpb24iLCJkZWZhdWx0LXJvbGVzLWlkbSJdfSwicmVzb3VyY2
VfYWNjZXNzIjp7ImFjY291bnQiOnsicm9sZXMiOlsibWFuYWdlLWFjY291bnQiLCJtYW5hZ
2UtYWNjb3VudC1saW5rcyIsInZpZXctcHJvZmlsZSJdfX0sInNjb3BlIjoib3BlbmlkIHBy
b2ZpbGUgZW1haWwiLCJlbWFpbF92ZXJpZmllZCI6ZmFsc2UsIm5hbWUiOiJKb2huIERvZSI
sInByZWZlcnJlZF91c2VybmFtZSI6Im5ldy11c2VyIiwiZ2l2ZW5fbmFtZSI6IkpvaG4iLC
JmYW1pbHlfbmFtZSI6IkRvZSIsImVtYWlsIjoibmV3LXVzZXJAaWRtLnJ1In0.Sn9OkTQoe
o1ANmQlVOLibo0cfYqSWLemlsuNTAbTbAxWIAOhhtTkqqnWGCxBX6VZmBNnUEyDPMN_wHBF
K2mOb22PvgXQ_Slm3tOxZRbrRMBiPouSJRK9rcMGe0TP1MLJ_impPJLIXoTwXNBbuNVeCnG
UOJ0P7D_mUUippXHXzoulau_rG1HuVH1uQi7UJWc81s5-
j_LhyUcrzzchCU8rx2YMNiWAxby1r9zI8Dnq4QTF5o6xxp32sK_oEGY6o_OXdSohLEWeeY9
3HSx0s9yfkVxip42oSzo0KdTs8e7tiqBZISSjvHQf3UMptmnPHAp5RPto7-
DRlFlQ2ELnlRvuCA
```


23

24

DECODED HEADER

JSON CLAIMS TABLE COPY


```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "RWkpAVmnitxQvhL0UxaO2JrSvoklOmjlcWRfWFFbqTg"
}
```


DECODED PAYLOAD

JSON CLAIMS TABLE COPY


```
{
  "exp": 1746345615,
  "iat": 1746345315,
  "auth_time": 1746345314,
  "jti": "onrtac:0c43d816-2267-43ec-b99c-90cd2c6db1b5",
  "iss": "http://localhost:9990/realms/idm",
  "aud": "account",
  "sub": "8b43704f-2c4c-4f3c-b364-026a920a6018",
  "typ": "Bearer",
  "azp": "idmapp",
  "sid": "65ef1f90-1849-4172-9bfc-f705a9ce6a7e",
  "acr": "1",
  "allowed-origins": [
    "https://www.keycloak.org"
  ],
  "realm_access": {
    "roles": [
      [redacted]
    ]
  },
  "resource_access": {
    "account": {
      "roles": [
        "manage-account",
        "manage-account-links",
        "view-profile"
      ]
    }
  },
  "scope": "openid profile email",
  "email_verified": false,
  "name": "John Doe",
  "preferred_username": "new-user",
  "given_name": "John",
  "family_name": "Doe",
  "email": "new-user@idm.ru"
}
```


На этом запуск в докере и настройка Keycloak закончены.

Задачи на сегодня:

1. Запустить у себя в докере Keycloak.
2. Декодировать токен, полученный с его помощью.
3. В комментарий к задаче скопировать всё содержимое секции Payload токена.